

DCLT Portal — Conceptual Architecture

Last updated: May 5, 2026. Supersedes the previous version. This document is the upstream authority for the DCLT Portal data model, vocabulary, and disciplines. Implementation extends but never overrides what's here.

Purpose

This is the canonical conceptual reference for the DCLT Portal. It defines the data model, the disciplines that govern it, and the vocabulary used to describe it. Two audiences:

Project conversations. This document primes work in this project. New threads inherit its model without re-deriving it.

Claude Code. This document is the upstream authority for any schema or implementation work. Claude Code consumes it and implements directly against the codebase. There is no separate Technical Architecture document; the codebase is the implementation.

When implementation surfaces gaps or contradictions, the resolution path is to update this document first, then implement. The codebase never silently diverges from the model.

Scope

This document defines:

- The Tag / Layer dichotomy and the authorship test that separates them
- Layer source types (External / Derived / Dynamic) and their refresh disciplines
- Tag dimension structure (state space, default, applicability, transitions)
- Composition rules between Tags and Layers, and cycle prevention
- Filtering semantics: curation, idioms, combination, cross-node, applicability
- Node types, including links as relationships between nodes
- Surfaces: tab inventory, generic vs bespoke surfaces
- Campaigns: per-dimension workflow surfaces driven to terminus
- Naming canon

- Worked examples grounding the abstractions

This document does not define schema DDL, file layouts, ingest implementation, materialization mechanics, evaluator design, query patterns, or UI surface implementation. Those live in the codebase.

Guiding principles

Eight principles drive the model. The first four are commitments — what the architecture exists to support. The last four are disciplines — how the architecture stays coherent as it grows.

Hard data is gettable. What's been ingested was hard-won; what's asked for next is probably feasible. The architecture encourages incremental sourcing rather than presuming everything is known up front.

The interface serves the question. Same data, different lenses, depending on what's being decided. Filtering and presentation are first-class concerns, not an afterthought on top of storage.

Human judgment is captured, not lost. Tags, notes, and determinations from people who know the ground are first-class data, distinct from machine guesses, with full provenance.

Lineage is non-negotiable. Every answer traces to its source. The split between machine-extracted and human-confirmed is preserved in every read.

Authorship is the seam. Every design question routes back to: who authored this value? That's what determines Tag vs Attribute, what makes the append-only discipline non-negotiable, why "system tag" is banned vocabulary. Contested calls have a clear test.

Re-ingest must not disturb human work. The most important operational invariant. Drives the Tag/Layer split, the append-only log, applicability-as-projection (so attribute changes don't write tag rows), and the reference.db / transactions.db separation. If a proposed change can disturb human work on re-ingest, the proposal is wrong.

One way to express each shape. When two patterns could express the same thing, pick one. Cross-node tag filters are existential; counts are Dynamic Layers. Applicability is a query projection, not a state. Dimensions live in code; filter curation lives in configuration. Picking one keeps the model legible as it grows.

Complexity follows demand. The model handles the cases in front of us cleanly and leaves room for the cases that haven't shown up. Edge cases get positions when a real request surfaces them, not when they're imagined. A simple rule that covers the current need beats an elaborate rule that anticipates needs that may never arrive. When extension is required,

the resolution path is the one already defined: name the gap, take a position in this document, propagate to the codebase.

The loop

The architecture exists to support a four-step cycle:

1. **Surface what's there.** External Layers source from feeds; Derived Layers calculate from those.
2. **Ask what we could do with it if accurate.** Dynamic Layers and queries pose the question.
3. **Let people who know the ground convert guesses into determinations.** Tag dimensions are the workflow.
4. **Re-ask with stronger footing.** Each pass, the latest fold answers with more credibility.

Re-ingest never disturbs human work — it lives in Tags. Changing a Layer formula never disturbs history — Layers don't have history. Changing a calculation never disturbs storage — Derived Layers re-materialize on next ingest, Dynamic Layers re-evaluate on next read, neither touches transactions.db.

Where this lives today

Single-server portal, VPS-hosted, GitHub-deployed. A local processing pipeline produces a read-only reference.db containing External and Derived Layers, deployed alongside the application. transactions.db lives on the server, append-only, never overwritten by deployments. Surfaces are organized as a fixed set of tabs (see Surfaces); the Campaigns tab is the primary decision surface, with Parcels, Registry, and Town Documents as generic browser surfaces.

The dividing line

Authorship in the portal is the only test.

Attribute. Not authored by users in the portal. Sourced from external feeds, derived by calculation from other Layers, or computed dynamically using Tag inputs. Lives in a Layer.

Tag. Authored by users in the portal. Default-seeded so the dimension is queryable from day one, then assorted by users as work progresses. Lives in transactions.db.

The seam is authorship of the current value, not authorship of the seed. A Tag whose default is a system suggestion is still a Tag — the seed is a proposal, the user's assortment

is the verdict, and human authorship beats the seed regardless of timing. An Attribute whose calculation depends on user-authored Tags is still an Attribute — no user asserts the calculated value directly.

Layers (attributes)

Every Attribute belongs to a Layer. A Layer has two classifying axes.

Source type

Three values.

External. Sourced from a feed. Assessor exports, MassGIS layers, Registry OCR output, Dave's ownership spreadsheet.

Derived. Calculated from other Layers (External or Derived only). Materialized in reference.db. Refreshed on ingest.

Dynamic. Calculated using at least one Tag fold as input. Never materialized. Evaluated at read time.

A Layer is Dynamic if it reads any Tag fold, Derived otherwise. Classification follows the inputs, not the intent. Adding a Tag input to a Derived Layer reclassifies it — code review catches the change.

Node type

The output key. Parcel, Document, ParcelDocumentLink, or any future node type. Future node types are added by code change, not configuration.

A Layer that aggregates document data into per-parcel rollups is a parcel-typed Layer with document inputs — node type follows the output, not the inputs.

Links are a node type. A link's target_id is a composite of the related nodes' identifiers (e.g., parcel_id + document_id). Suspected links are produced by machine processes; user adjudications on those suspicions are Tags on the link node. Treating links as a node type keeps the model uniform — link Tags and link Attributes follow the same rules as everything else.

Naming

`<NodeType><Subject>` for derived and dynamic Layers, e.g., `ParcelFootprintRatio`, `DocumentKeywordScores`, `ParcelCoverageRollup`, `SuspectedParcelDocumentLink`. External Layers may inherit names from their source schema.

Storage and refresh

Source type	Storage	Refresh
External	reference.db	Ingest
Derived	reference.db (materialized)	Ingest
Dynamic	In memory (formula only)	Per read

If a Dynamic Layer becomes a performance problem, the escalation is an overnight materialization job — caching, not a storage commitment. The Layer's classification doesn't change; only its read path does.

Tags

A Tag is a dimension for assorting nodes (parcels, documents, links, others) with a finite state space. Users move records through that space. The current state on a dimension is the latest-fold of the tag log for that (target_type, target_id, dimension).

Mechanics

Append-only log in transactions.db. Every event records target (target_type, target_id), dimension, state, user_id, server-assigned monotonic seq, client created_at, optional note. No UPDATE, no DELETE — corrections append a new event. Latest row wins per (target_type, target_id, dimension). Full history preserved and queryable.

A database-level trigger rejects UPDATE and DELETE on the tag table. Application code paths must never construct such statements.

Dimension definition

Each Tag dimension is defined by six properties, all in code:

1. **Name.** `CoverageDetermination`, `IdentityResolution`, `Article97Determination`, `LinkAdjudication`, `DCLTOwnership`.
2. **Node type.** Which target type the dimension applies to.
3. **State space.** Closed set of valid states. Includes `Unconfirmed` if the dimension is a decision-pending workflow. Does not include `Not Applicable` — applicability is a separate axis (see Filtering).
4. **Default rule.** How new records seed. Constant (usually `Unconfirmed`), or a function of

attributes. Default rules read External or Derived Layers only — never Dynamic — to keep the dependency graph acyclic.

5. **Applicability rule.** Which records the dimension is meaningful for. May depend on attributes. `IdentityResolution` is dormant when `IdentityState = OK`.

`Article97Determination` only applies to documents with a keyword hit above threshold. `LinkAdjudication` only applies to suspected links.

6. **Allowed transitions.** Usually any-to-any. Sometimes constrained by an attribute:

`IdentityResolution` permits {ADB Add, GIS Remove} when `IdentityState = GIS-only`, {GIS Add, ADB Remove} when ADB-only.

Adding a Tag dimension is a small commit, not a runtime config change. Dimensions are versioned with the code; the log preserves history through definition changes.

Whether a dimension appears in filter chrome and where it groups in detail panels is a separate concern, handled by configuration (see Filtering).

Hygiene journey

`Unconfirmed` is the work queue. Filter to it, process, watch coverage Layers climb. Derived or Dynamic Layers like “% adjudicated” and “% resolved” make progress visible across the inventory.

Naming convention

- `-Determination` for categorical verdicts where users decide what an observation means.
- `-Resolution` for dispositions on a discrepancy.
- `-Adjudication` for verdicts on machine-suspected relationships (links).
- Plain name where the dimension itself is a workflow with no parallel attribute (`DCLTOwnership` when the source is users-marking-in-the-portal).

One suffix per Tag shape keeps the model legible as the dimension count grows.

Composition

Layers and Tags compose by reading from each other under constrained rules.

Layers can read Tags. A Layer that reads any Tag fold is Dynamic. This is the only way a Tag’s current state influences a calculated value.

Tag default and applicability rules can read Attributes. Default seeds and applicability

conditions may depend on External or Derived Layers. They may not depend on Dynamic Layers — that would create a cycle (Tag → Dynamic Layer → Tag).

Tags do not read Tags. A Tag's state space and seed are independent of other Tags' folds. If a workflow seems to require otherwise, the right shape is a Dynamic Layer that combines the Tags, queried wherever the combination matters.

Attributes and Tags never write to the same row. They live in different stores with different keys. The "human beats system" fold rule is automatic, not enforced — there is nothing for system writes to overwrite.

Filtering

Filtering is a first-class concern. Three axes are visible to the user: tag fold state, attribute value, and applicability. The first two are filter selections; the third is an automatic projection from the dimension's applicability rule.

Filter curation

Each node type carries more dimensions than belong in any one filter chrome. Which Tags and Attributes appear in the filter UI for a given node type is configuration, not code. The same goes for grouping in detail panels. Filter curation is editable without redeploy and lives in transactions.db alongside other portal state.

The split: code defines what a dimension means (the six properties for Tags, the formula and source for Attributes); configuration decides where it appears in the UI. This split applies to generic surfaces (see Surfaces). Bespoke surfaces govern presentation in code — configuration around them is limited to enable/disable, ordering, and similar metadata.

Curatable per node type: which Tag and Attribute dimensions render as filter chips, how they group in detail panels, and the value selections that deep-link entry points (e.g., a left-nav click on a Suitability category) pre-populate. Default selections belong to entry points, not to the chips themselves — a chip in filter chrome holds no state until the user opens its picker and chooses. Deep-link payloads may set both the entering dimension's selection and a companion dimension's selection (e.g., `Suitability = Likely` paired with `Determination = Unconfirmed`) so that gated workflows land in the work queue by default.

Filter chip behavior

Filter chrome renders one chip per curated dimension. A chip is a handle, not a filter — clicking opens the dimension's picker; no rows are filtered until the user selects a value and confirms. Picker shape follows the dimension's idiom (see below): multi-select for limited-set Attributes and Tag state spaces, range slider for numeric Attributes, two-axis (state

multi-select + separate Not Applicable affordance) for Tag dimensions with conditional applicability.

Chip-click and deep-link entry are different actions. Chip-click opens an empty picker. Deep-link arrives with selections pre-populated per the entry point's curated payload. Same chip, same picker; the difference is what populates it on arrival. Lifting a deep-link selection is symmetric with applying any other selection — open the chip, edit the picker.

Tag filters

When a user opens a Tag chip, the picker is a multi-select over the dimension's state space. A record matches if its current fold is among the selected states and the dimension is applicable to it. If the dimension's applicability rule is conditional, the picker also offers `Not Applicable` as a structurally separate affordance — visually distinct from state choices, since it filters on a different axis.

For dimensions whose applicability rule is "all records," `Not Applicable` is omitted (the result would always be empty). For dimensions like `IdentityResolution`, `Article97Determination`, or `LinkAdjudication`, `Not Applicable` is present and meaningful.

Within a single Tag filter, state values are unioned (OR). Selecting both states and `Not Applicable` returns the union.

Attribute filters

Filter idiom follows attribute shape:

- **Limited-set values.** Multi-select picker.
- **Numeric continuum.** Range slider.
- **Boolean.** Degenerate two-option multi-select.
- **Free-form text.** Not filterable; appears in detail panels only.

Attributes can have applicability rules just as Tags do. A coverage ratio is only meaningful for parcels with known footprint and acreage. The `Not Applicable` affordance applies the same way.

Combining filters

Selected filters across dimensions intersect (AND). A record matches if it satisfies every selected filter clause. Within a single filter (a tag's state multi-select, an attribute's range), values are unioned (OR).

Order of selection doesn't matter. The filter set is a conjunction; the user assembles clauses, the system intersects.

Cross-node filters

A filter on one node type can include clauses on dimensions of a different node type, joined by a relationship. The first case the model addresses: parcel filters with document-scoped clauses, joined by parcel-document links.

Semantics are existential: a parcel matches if any linked document satisfies the document-scoped clause. Counts and thresholds belong in Dynamic Layers, exposed as parcel attributes — not in the cross-node filter itself.

The link adjudication is part of the join. By default, only links with `LinkAdjudication = Confirmed` participate. Unadjudicated links are noise; rejected links are wrong. The default makes the filter a forcing function for adjudication work — a user who needs a parcel to satisfy a document-scoped clause must first confirm the relevant link. Engagement is the intent.

When the machine signal is strong enough that adjudication isn't yet feasible, the related Derived Layer (e.g., a suspected-link confidence score) is available as an attribute filter on the link. That's the documented escape hatch from the engagement gate.

Rollup denominators

Derived and Dynamic Layers that report dimension-level percentages count among the applicable population by default. `ParcelArticle97Rollup`'s "% confirmed" reads against documents with keyword hits, not against all documents — otherwise the hygiene number is structurally diluted. The `Not Applicable` count is reported separately when relevant, never folded into the active denominator.

Applicability flips

Applicability is computed at read time from current attribute values. A record can transition between applicable and not-applicable as upstream data changes. When that happens, the existing Tag fold is preserved in history and remains queryable, but the record falls under `Not Applicable` for active filters until applicability is restored.

Example: a parcel was `GIS-only`, the user resolved it as `ADB Add`, and the next ingest updates the Assessor's database so `IdentityState` flips to `OK`. The fold remains `ADB Add` in history; the active filter shows the parcel as `Not Applicable`. The resolution did its job and the dimension stepped back.

Surfaces

The portal renders the model through a fixed set of tabs. Each tab has a defined role; surfaces split into generic and bespoke.

Tab inventory

- **Overview.** Public, static. Describes what the site is and how to get access. No login required, no model content, no rollups.
- **Campaigns.** Authenticated. Primary decision surface. Bespoke per-Campaign components (see Campaigns).
- **Parcels.** Authenticated. Generic browser over the Parcel node type.
- **Registry.** Authenticated. Generic browser over registry-of-deeds Documents, with viewer.
- **Town Documents.** Authenticated. Generic browser over town meeting Documents, with viewer.
- **Assets.** Authenticated. Reference inventory of ingested data (External and Derived Layers) plus active Tags. Source-of-record surface, not a workflow surface.
- **Admin.** Authenticated, restricted. Filter curation editing, Campaign configuration (visibility, ordering, lead assignment), user management.

Generic vs bespoke surfaces

Two surface disciplines, two configuration disciplines.

Generic surfaces render a shared component over a shared filter chrome. Curation decides which dimensions appear as chips, how detail panels group dimensions, and what deep-link payloads pre-populate. Curation is configuration, edited via Admin. Parcels, Registry, Town Documents, and Assets are generic surfaces.

Bespoke surfaces render a per-workflow component, code-defined. Curation isn't a chip set — it's the layout itself: which evidence to show, how to present it, what side affordances to provide. Code captures this naturally; configuration would have to become a layout DSL to express it. Campaigns is the first bespoke surface.

The split is a softening of the earlier rule that code defines meaning and configuration decides UI placement. That rule still holds for generic surfaces. For bespoke surfaces, code governs both meaning and presentation. Configuration around bespoke surfaces is limited to enable/disable, ordering, and lead assignment — not layout.

Decision affordances concentrate on Campaigns. Generic browser surfaces retain inline decision affordances for opportunistic assortment (a user browsing parcels can still write a fold), but the chrome no longer foregrounds them.

Campaigns

A Campaign is a workflow surface that drives a single Tag dimension to terminus. Terminus is when no records remain in the work queue — Unconfirmed = 0 within the applicable population.

Definition

A Campaign is a tuple:

- **Node type.** Which target type the Campaign assorts.
- **Primary Tag dimension.** The dimension being driven to terminus. Exactly one per Campaign.
- **Applicable-population predicate.** Inherited from the dimension's applicability rule. Defines the denominator.
- **Supporting curation.** Which Attributes and companion Tag dimensions appear in the Campaign's detail layout. Code-defined.
- **Rollup Layer.** Dynamic Layer producing Campaign-level percentages by state across the primary dimension's state space. Drives the summary panel and any cross-Campaign progress views.
- **Deep-link payload.** Default selections for the Campaign's entry point — typically primary dimension = Unconfirmed.

One dimension per Campaign, by definition. Multi-dimensional workflows decompose into multiple Campaigns or into a composite Dynamic Layer with no Campaign at all.

Workstreams are not 1:1 with Campaigns: some workstreams produce one Campaign, some several, some no Campaign (composite Layer instead), some neither (expert-packet views are a separate surface, not yet defined).

Code vs configuration

The Campaign descriptor (the tuple above) and the per-Campaign component live in code. Adding a Campaign is a commit, same shape as adding a Tag dimension — and Campaigns and dimensions tend to land together.

Configuration governs three properties per Campaign, stored in transactions.db, edited via

Admin:

- **Visibility.** Shown, hidden, or de-emphasized. A Campaign committed in code but not enabled does not appear in the Campaigns tab. Lets Campaigns ship dark, get tested, and turn on without redeploy.
- **Ordering.** Position in the Campaigns tab landing and within the tab's left nav.
- **Lead.** DCLT contact for the Campaign.

Tab structure

The Campaigns tab has two levels.

Landing. Summary panels, one per enabled Campaign. Each panel shows:

- **Scope.** Applicable-population size and node type.
- **Assortment.** Rollup Layer output across the primary dimension's state space, including Unconfirmed.
- **Lead.** DCLT contact.
- **Last Tag change.** Most recent fold in the Campaign's primary dimension. $\text{MAX}(\text{seq})$ over the dimension in transactions.db; null when no folds exist beyond seeded defaults. This is the canonical signal of human activity on the Campaign — distinct from last ingest, which doesn't move it.

Selecting a panel navigates to the Campaign workspace, with the deep-link payload applied.

Workspace. Per-Campaign component. Typical structure: work queue (filtered list driven by the deep-link payload), detail panel (selected record with supporting curation), in-context affordances for writing the determination. Layout, evidence presentation, and side affordances are Campaign-specific and code-defined.

Cross-Campaign composition

Campaigns produce Tag folds. Downstream consumers — other Campaigns, Layers, external assertions — read those folds as the dimension's current state. When the consumer needs a transformed value (a typed boolean, a categorical aggregate), a Dynamic Layer wraps the fold. Direct fold reads are fine when no transformation is needed.

A downstream Campaign whose applicable-population predicate depends on an upstream Campaign's verdict reads the upstream dimension via Dynamic Layer, never by Tag-reads-Tag (forbidden by Composition). Subdividability over Coverage is the canonical example:

`SubdividabilityDetermination` 's predicate is " `CoverageDetermination` =

`Underdeveloped`," resolved through a Dynamic Layer.

Hygiene rollups

The Campaigns tab landing is the canonical hygiene surface. Per-Campaign rollup Layers aggregate to the landing; cross-Campaign progress is visible at a glance. The Overview tab is public and static and does not host hygiene rollups.

Examples

Coverage

`ParcelFootprintRatio` is a Derived Layer (footprint area ÷ parcel area, computed from External Layers).

`CoverageDetermination` is a Tag.

- States: {Unconfirmed, Undeveloped, Underdeveloped, Developed}
- Node type: Parcel
- Default: Unconfirmed
- Applicability: all parcels
- Transitions: any-to-any

`ParcelCoverageRollup` is a Dynamic Layer producing inventory-level percentages by determination state — drives the Coverage Campaign's summary panel on the Campaigns tab landing.

Identity

`IdentityState` is a Derived Layer with states {OK, ADB-only, GIS-only}, calculated each ingest from External Layers (Assessor and GIS presence).

`IdentityResolution` is a Tag.

- States: {Unconfirmed, ADB Add, ADB Remove, GIS Add, GIS Remove}
- Node type: Parcel
- Default: Unconfirmed
- Applicability: only when `IdentityState` ≠ OK
- Transitions: constrained by `IdentityState` (GIS-only → {ADB Add, GIS Remove}; ADB-

only → {GIS Add, ADB Remove})

Article 97

`DocumentArticle97KeywordScore` is an External Layer (OCR keyword scorer output).

`Article97Determination` is a Tag.

- States: {Unconfirmed, Confirmed, Denied}
- Node type: Document
- Default: Unconfirmed when keyword score is above threshold; dimension inapplicable otherwise
- Applicability: documents with keyword score above threshold
- Transitions: any-to-any among the three states

`ParcelArticle97Rollup` is a Dynamic Layer aggregating document determinations to the parcel level (count confirmed, count unconfirmed, etc.). Denominator is applicable documents.

Parcel-document link

The link is a node type; suspected links are an Attribute; user adjudications are a Tag.

`SuspectedParcelDocumentLink` is a Layer (External or Derived depending on source). Inputs vary by document source: registry deeds carry book/page references that can be joined against the Assessor's database; town meeting documents require candidate generation that scans full text against parcel identifiers. The Layer is tuned for high recall — false negatives are worse than false positives, because a user can dismiss a wrong suggestion but cannot see a missed one. Per-link attributes record the evidence basis.

`LinkAdjudication` is a Tag.

- States: {Unconfirmed, Confirmed, Rejected}
- Node type: `ParcelDocumentLink` (`target_id` is a composite of `parcel_id` and `document_id`)
- Default: Unconfirmed for every suspected link
- Applicability: only suspected links — there is no point adjudicating a relationship no machine ever proposed
- Transitions: any-to-any

Cross-node filters between parcels and documents read this dimension via the confirmed-only default described under Filtering.

DCLT ownership

Two cases, depending on the source of truth.

If users mark ownership directly in the portal: `DCLTOwnership` is a Tag with states {Unconfirmed, Owned, Not owned}, applicability all parcels.

If Dave's spreadsheet is the source of truth and arrives by feed: `ParcelDCLTOwnership` is an External Layer with states {owned, not listed} and an as-of date. A Tag may still exist on top to record portal-side overrides when a user notices the spreadsheet is wrong — same shape as Article 97 (attribute observes, tag overrides).

Migration notes

The current implementation is expected to have:

- A single tags surface mixing system-written and user-written rows.
- Dimensions whose seeds are written by ingest and treated as authoritative until overwritten by a user.
- Calculated values co-mingled with stored ones, distinguished informally if at all.
- Filter curation embedded in code rather than configuration.
- Parcel-document link adjudication in a separate, ad-hoc table.
- The transactions store named `dclt.db` rather than the canonical `transactions.db`.
- Filter chips that auto-apply on click rather than opening a picker.
- Decision affordances spread across Parcels, Registry, and Town Documents tabs rather than concentrated on a Campaigns tab.

The refactor:

1. **Inventory existing tag dimensions.** Classify each as Tag (user-authored in portal) or Attribute (system-authored). The "system tag" category disappears.
2. **Move attribute dimensions into Layers.** Decide source type (External / Derived / Dynamic) per dimension based on inputs.
3. **Restrict the Tags table to portal-authored events only.** Add the UPDATE/DELETE trigger if not present. Audit existing rows; system-written rows migrate to their Layer's

storage.

4. **Rebuild seeds-from-system as default rules on Tag dimensions.** The default reads the relevant Layer at fold time rather than being written into the log at ingest.
5. **Rename `dclt.db` to `transactions.db`** to align with the canonical name. Update all configuration keys and access functions accordingly.
6. **Introduce the link node type.** Migrate parcel-document link adjudications into the Tag table under `target_type = ParcelDocumentLink`. Suspected links become an Attribute in a Layer.
7. **Move filter curation into configuration.** Curatable: which Tags and Attributes appear in each node type's filter chrome, which appear in detail panels, how detail panels group dimensions, and the value selections that deep-link entry points pre-populate.
8. **Audit read paths.** Every place that reads a "tag" today, decide whether the consumer wants the Tag fold (decision state) or the Layer value (observation). Update accordingly.
9. **Audit filtering UIs.** Replace any filter that treats applicability as a state value with the two-axis pattern (state multi-picker + separate Not Applicable affordance). Add attribute filters per the idioms above. Add cross-node filtering for parcel-by-document with confirmed-link-only default. Convert filter chips to handle-then-picker behavior — chips never auto-apply on click.
10. **Stand up the Campaigns tab.** Implement the Campaign descriptor registry (in code), the Admin configuration surface (visibility, ordering, lead), and the summary-panel landing. Register active workstreams as Campaign descriptors as their primary Tag dimensions are defined. Move decision affordances off the generic browser tabs into Campaign workspaces; leave inline assortment available on the browser tabs but de-emphasized.

The `reference.db` / `transactions.db` split survives unchanged. What changes is what goes on each side of the line, and what the application reads to get current state.

Naming canon

Claude Code and downstream documents should use these terms verbatim. Synonyms and parallel vocabulary fragment the model.

Categories. Tag, Layer, Attribute, Dimension.

Layer source types. External, Derived, Dynamic.

Node types. Parcel, Document, ParcelDocumentLink. Future node types added by code

change, not configuration.

Tag mechanics. fold (verb and noun), latest-wins, append-only, target (target_type, target_id), seq, applicability, applicable, Not Applicable.

Dimension parts. state space, default rule, applicability rule, allowed transitions, transition.

Tag suffixes. -Determination , -Resolution , -Adjudication , plain workflow name.

Filtering. filter curation, filter chrome, filter chip, picker, deep-link entry point, deep-link payload, detail panel, multi-select, range slider, two-axis, cross-node filter, existential, confirmed-link-only.

Surfaces. tab, generic surface, bespoke surface. Tab names: Overview, Campaigns, Parcels, Registry, Town Documents, Assets, Admin.

Campaigns. Campaign, primary Tag dimension, applicable-population predicate, supporting curation, rollup Layer, summary panel, workspace, scope, assortment, lead, Last Tag change, terminus.

Stores. reference.db, transactions.db.

Pipeline. ingest, materialize, evaluate, refresh.

Banned vocabulary:

- "system tag," "user tag," "automatic tag" — the old conflation. Use Tag (always user-authored) or Attribute (anything system-sourced or calculated).
- "calculated attribute" as a stored on-disk artifact other than a materialized Derived Layer. If it reads Tags, it's Dynamic and lives only as a formula.
- "update tag," "delete tag" — Tags are append-only. Corrections are new events.
- "Not Applicable" as a state in any state space. It's a query-time projection.
- `dclt.db` — superseded by `transactions.db`.
- "auto-applied chip," "default chip state" — chips hold no state. Defaults belong to deep-link entry points.
- "Project," "Loop," "Objective" as synonyms for Campaign — the term is Campaign.
- Multi-dimensional Campaign — a Campaign drives exactly one Tag dimension to terminus.
- Hygiene rollups on Overview — Overview is public and static. Rollups live on the Campaigns tab landing.

- “last updated,” “last activity” as labels for the human-activity signal on a Campaign — the term is Last Tag change.

Implementation

The codebase is the implementation companion to this document. It is downstream — it implements what’s defined here, in the vocabulary defined here. There is no separate Technical Architecture document; Claude Code works directly from this CA against the code.

Implementation covers:

- Schema DDL: tables, columns, types, primary and foreign keys, the WORM trigger
- Indexes supporting the latest-wins fold and history retrieval
- File and module organization
- Ingest pipeline: fetching, normalization, joining against the parcel spine, OCR processing
- Derived Layer materialization: order, dependency resolution, rebuild semantics
- Dynamic Layer evaluation: where formulas live, how they’re invoked, caching strategy
- Tag write path: validation against state space, applicability check, transition check
- Filter configuration: schema (including chip placement, detail-panel grouping, and deep-link payloads), editing surface, change auditing
- Query patterns combining attribute filters, tag fold filters, applicability projections, and cross-node joins
- Performance: where caching applies, escalation criteria for materializing a Dynamic Layer overnight
- Migration mechanics for moving from the current implementation
- UI surface implementation, including the filter chip handle-then-picker behavior, the two-axis Tag filter pattern, attribute filter idioms, and cross-node filter chrome
- Tab routing and the seven-tab structure
- Campaign descriptor registry: where descriptors live, how they’re registered, how Admin configuration rows (visibility, ordering, lead) join to them
- Per-Campaign component organization: module boundaries, shared primitives, evidence-presentation idioms

- Campaigns tab landing: summary panel rendering (scope, assortment, lead, Last Tag change) and deep-link payload application on selection
- Last Tag change query against transactions.db
- API shape, deployment, environment

Implementation must not:

- Redefine, rename, or split the categories defined here (Tag, Layer, Attribute, Dimension)
- Introduce parallel vocabulary or new top-level abstractions without first proposing them as a CA change
- Decide composition rules that contradict the rules in this document
- Add states like "Not Applicable" to a dimension's state space
- Materialize Dynamic Layers as a default storage strategy (caching is acceptable; storage commitment is not)
- Allow UPDATE or DELETE on Tags under any circumstance
- Embed filter curation in code
- Render filter chips that auto-apply on click
- Render hygiene rollups on the Overview tab (Overview is public and static)
- Define a Campaign that drives more than one Tag dimension
- Embed Campaign visibility, ordering, or lead assignment in code (these are configuration)

When implementation needs more than this document provides:

The codebase may surface gaps: examples that don't fit cleanly, composition patterns this document didn't anticipate, applicability or transition rules that need a position. The resolution path is to propose an update to this CA, get it accepted, and then implement. The codebase does not extend the model unilaterally.

Evolution

This document is canonical for the model but not frozen. Implementation will surface things the model didn't anticipate. When that happens:

1. The gap or contradiction is named explicitly.
2. A position is taken in this document — new section, amended rule, additional example.

3. The codebase is updated to match.

This document supersedes prior conceptual architecture documents. Future revisions are tracked in version control alongside the codebase.